EXAMINATION

Luleå University of Technology

Course: ***D7024E***
Course name: ***Distributed and Mobile Computing Systems***
Date: ***2024-10-28***
Time: ***13.00-17.00, online home exam***
Aid: ***WWW and Generative AI (e.g., ChatGPT, Bard) (no interactive communication with other people)***

Teacher on duty (complete telephone number): Johan Kristiansson, 070-3632912 johan.kristiansson@ltu.se	***Teacher on duty (complete telephone number):*** Olov Schelén, 0920 492237 olov.schelen@ltu.se
Teacher on duty (complete telephone number):	***Teacher on duty (complete telephone number):***

Grade scale: ***50% grade 3, 75% grade 4 and 90% grade 5***	Note that this grade is for the exam. The exam is supposed to be easy enough so that anyone that followed the course properly should pass it. The grade for the course (including all assessment criteria) may be different from the grade of the exam.
Total number of questions and score: ***total score 42***	
Other information: This is an individual home exam.	Note: No interactive cooperation with other people is allowed. Information can be searched on the Web and with generative AI (e.g., ChatGPT, Bard, etc.). Please provide references when such information is used in your answers.

General instructions

You shall formulate answers with your own words; if you use generative AI a personal reflection must be included. In any case reference your primary sources (e.g., URL). Your ***solution must be uploaded in the Canvas D7024E room under Assignments***, at the latest at 17:00 (unless you have gotten approval for extra time). ***Please put your name both in the filename and on the first line of your document.***

After examination

Examination results are posted on Canvas within 4 weeks after the examination.

1. **Mixed questions on distributed systems (6p)**

   These are multi-choice questions numbered a-l, with three alternatives. Please answer "true", "false" or “depends” to each of the following statements. If your answer is “depends” you must explain briefly to get credits. The answer “true” or “false” requires no explanation unless you want to explain some assumed conditions. If you have identified a corner case and provide a good explanation, we can accept an answer that is different from the expected one. Each question is worth 0.5p

a. If every low-level write is atomic, then all concurrent transactions are automatically serializable.

b. ACID in databases typically requires that all commands inside a transaction must be executed before another transaction can start.

c. Non-Byzantine consensus is typically designed to be horizontally scalable.

d. A DHT is an overlay network with additional data storage functionality.

e. Non-Byzantine consensus based on pairwise messaging requires N = 2f+1 nodes to handle maximum f nodes failing.

f. One reason Byzantine-fault-tolerant consensus is difficult is that there can be dishonest nodes, but some honest nodes could also crash.

g. A distributed system based on vector clock can ensure eventual consistency if RPC messages are mutable.

h. CRDTs suffer from drawbacks such as interleaving (where concurrent edits can result in unintuitive ordering of operations) and thus requiring a central server to resolve conflicts.

i. Logical clocks be used to create a total order of events in a distributed system.

j. Consistent hashing means that only a constant fraction of keys are moved between nodes.

k. It is possible to develop a distributed system that is both consistent and available.

l. Operational Transformation (OT) system guarantee that all users always see the same document state immediately, even when edits happen concurrently.

2. Gaming and blockchain (10p)

a) Choose an online multiplayer game genre we discussed in class (FPS, RTS, MMORPG, Fighting, turn-based, MOBA). Design the distributed backend for the game. What requirements are important, what system design considerations do you have to account for, and how do we ensure QoS? Explain the trade-offs involved in your choices and argue why your choices are correct in terms of QoS. Explain

how you can scale your system in case of increased demand. (6p)

b) Are Proof-of-Work (PoW) and Proof-of-Stake (PoS) consensus solutions? Why/how? Why not? Explain. (4p)

3. Virtualisation, microservices and compute platforms (12p)

c) Argue whether containers can (or should) replace virtual machines entirely in the next decade. Support your argument with reasoning grounded in the lecture material (e.g. hypervisors, namespaces, unikernels, hardware virtualisation) (3p)

d) Design a robust and scalable container orchestration framework focused on autoscaling of container workloads, following the same core principles as Kubernetes discussed in class. Ground your reasoning in distributed systems theories and algorithms covered in the lectures. (9p)
   a. Design a reconciliation loop that converges the system's current state towards a desired system configuration. The design must tolerate node crashes. Clearly state your assumptions about system behaviour and failure conditions.
   b. Detail the architecture of your framework, explaining how the control-plane components (e.g., API server, scheduler, autoscalers, state store) coordinate during scaling events and handle failures in a distributed environment.
   c. Draw a simple figure showing your system's data flow, decision flow, and failure management. Explain, step by step, what happens internally during a scaling event (up or down) is triggered.

4. Practical development and experiences (14p)

e) Compare process switching vs thread switching. Consider aspects you find important to compare and for those aspects describe how they are different and related trade-offs. You may want to include qualitative distinctions such as benefits, generality, security; performance aspects; software development aspects. (4p)

f) What are the key benefits of the channel abstraction for communication? Reason about it in different contexts from locally to globally. How can you benefit from channels in the lab assignment of this course. (4p)

g) GenAI tools can accelerate the development of distributed systems, but how can you be sure that the generated code behaves correctly? Explain how you would verify correctness and validate your Kadmelia implementation. Provide a short pseudocode example (from your lab work) showing how you could test or assert that a key aspect of your Kademlia implementation, such as node lookup or message routing, is correct with a large number of nodes. (6p)

Best Wishes!
Finally, please fill out the **online course evaluation in Canvas!**